

Machine Learning

Mikkel Moth Billing
202307989

4. september 2025

Indhold

Indhold	1
1 Linear model	2
1.1 Decision Trees	3
Entropy	4
Loss Function	5
1.2 Error	6
Loss Function Error	6
1.3 Linear Regression	6

1 Linear model

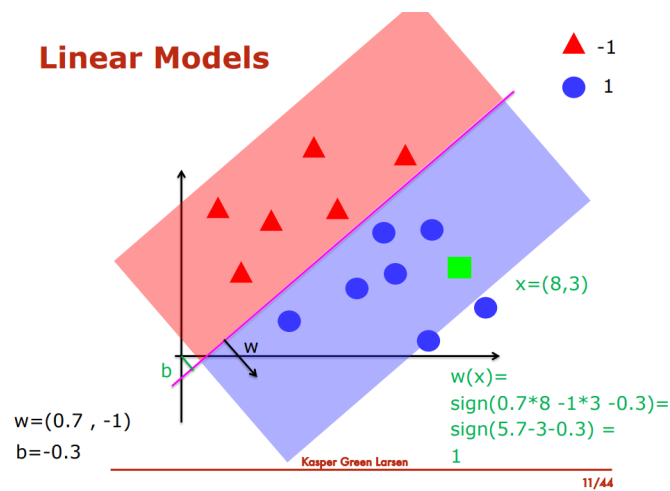
Hypothesis consists of the features vector:

$$x = \{x_1, \dots, x_d\}$$

and a bias b . If the linear combination of the weights and the input, and compare it to the bias, we can make predictions.

Example of mortgage, approve is 1 and decline is -1 . So we can predict that we approve

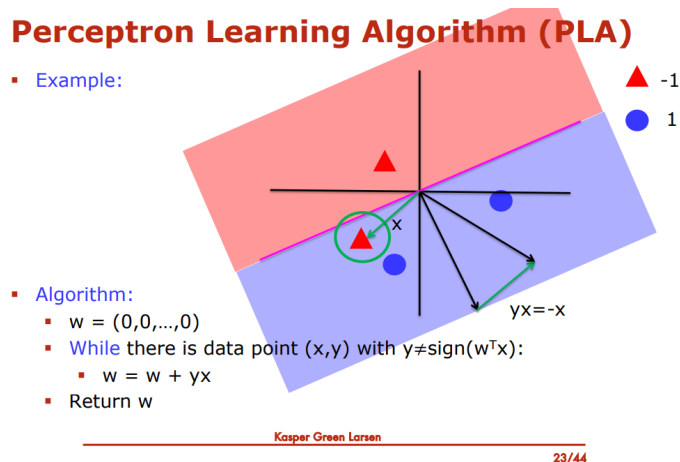
$$w_1x_1 + \dots + w_dx_d > -b$$



To find the parameters we can use **Perceptron Learning Algorithm (PLA)**.

Algorithm: While there is a data point (x, y) with

$$y \neq \text{sign}(w^T x) \rightarrow w = w + yx.$$



As we can see the algorithm can enter an infinite loop, as there might not be a linearly separable dataset.

A way to fix this is to use Pocket PLA which saves the most accurate hyperplane.

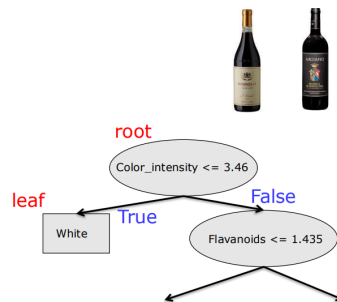
Pocket PLA

- Would like to still do something useful if not linearly separable!
- Pocket version: Remember best w in your pocket.
- Algorithm:
 - $w = (0,0,...,0)$
 - $bestScore = 0$
 - $bestW = w$
 - While you have more time, let (x,y) have $y \neq \text{sign}(w^T x)$:
 - $w = w + yx$
 - $score = \text{percentage of data correctly classified by } w$.
 - If $score > bestScore$:
 - $bestW = w$, $bestScore = score$
 - Return $bestW$

1.1 Decision Trees

Decision Trees

- Classifying wine.
 - Red, White, Rosé
- Predict(x):
 - Start at root node.
 - Test condition in root and descent to relevant child.
 - Upon reach leaf:
 - Return value stored at leaf.
- Node:
 - Question tests **one** feature against **one** value.

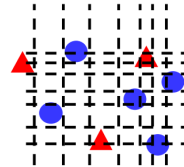
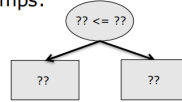


We would like to keep the decision tree small to not overfit to the training data. So we could find the best performing tree with k nodes. But this is **NP-hard**, so we rely on **heuristics**. With a greedy learning algorithm with $k = 2$:

Small Decision Trees

- Greedy Learning Algorithm.

- Decision Stumps:



- Find best decision stump:

- For each of d features:
 - For each of $n-1$ relevant splits:
 - Try that root, partition points into leaves, assign best leaves, compute accuracy.
- Use best split found.

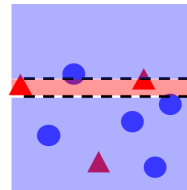
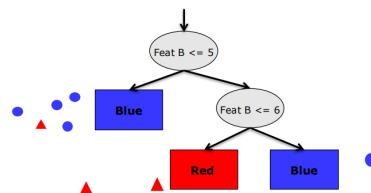
We test the tree on all possible split, this has $O(n^2d)$ runtime.

Another way to build a tree is to increase k to maximise the accuracy.

Small Decision Trees

- Final Tree:

$k=2$ example



Accuracy: 7/8

Entropy

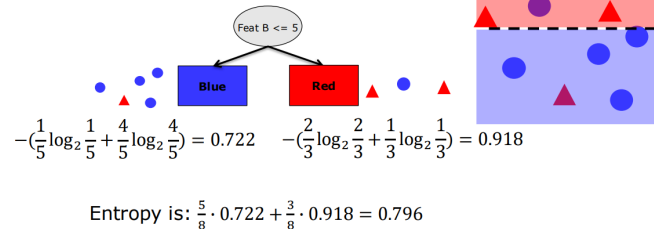
Another way to test decision tree is to base it on entropy. If we have n point with n_i as the numbers of points with label i . And $p_i = n/n_i$. Then entropy is:

$$H(S) = - \sum_{i=1}^k p_i \log_2 p_i.$$

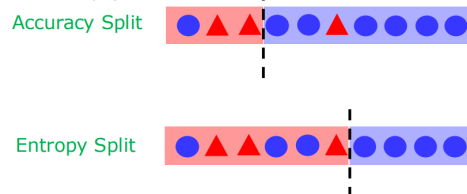
Entropy is a measure of uncertainty in distributions, high entropy is high uncertainty. $H(S)$ describes the expected number of bits it takes to describe the label of a random element drawn from S . If all elements in S are equal $H(S) = 0$. If there is an equal amount $H(S) = \log_2 k$.

Entropy

- Entropy of tree: $\sum_v \frac{|S_v|}{n} H(S_v)$



- May give different trees than using accuracy.
- 1-dimensional example, i.e. one feature.
 - Entropy prefers "pure" splits over accuracy.
 - Seem to work best in practice when constructing decision trees (larger than stumps).



Loss Function

A Loss Function weighs how much a mistake influences a model. We can then try to minimize the loss function over the training points. An example is the Least Square Loss Function:

$$L(y', y) = (y' - y)^2.$$

Small Regression Trees

- Greedy Learning Algorithm.
 - Start in simple case:
 - Our tree may only contain 1 node.
 - Root is a leaf.



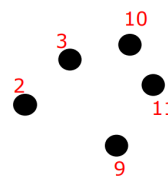
- What is best choice of root?

- The x that minimizes $\frac{1}{n} \sum_{i=1}^n (x - y_i)^2$ is the mean:

$$x = \frac{1}{n} \sum_{i=1}^n y_i.$$

Mean is $(2+3+9+10+11)/5=35/5=7$

Loss is $((2-7)^2 + (3-7)^2 + (9-7)^2 + (10-7)^2 + (11-7)^2)/5=14$.



1.2 Error

In-Sample Error (E_{in}) is the error on the training data, and is what we try to minimize. The Out-Sample Error (E_{out}) is the error on the unknown data, this is what we want to minimize. So we hope that $E_{in} \approx E_{out}$.

Loss Function Error

For the special Loss function 0-1 Loss, we know the error is

$$E_{in}(h) = \frac{1}{n} \sum_{i=1}^n 1_{h(x_i) \neq f(x_i)}.$$

The Out-Sample error is just the probability that $h(x) \neq f(x)$.

The Hoeffding inequality says that the probability of the two errors being further apart than some number ε is

$$P(|E_{in}(h) - E_{out}(h)| > \varepsilon) \leq 2e^{-2\varepsilon^2 n},$$

where n is training examples. If we have multiple hypotheses M we get:

$$P(|E_{in}(h) - E_{out}(h)| > \varepsilon) \leq 2Me^{-2\varepsilon^2 n}.$$

From this we can prove that minimizing $E_{in}(h)$ is a good strategy since for any other h' in H , we get that $E_{out}(h) \leq 2\varepsilon + E_{out}(h')$, so any other h' is at most 2ε better than h .

We can also use Hoeffding inequality to find out what sampling size we should use. If we want the difference in error to be less than 0.1 with 95% confidence, and $M = 1000$, we get:

$$2000e^{-2 \cdot 0.1^2 n} < 0.05 \Rightarrow n > 50 \ln(40,000) = 530.$$

If we are using an algorithm with infinite hypothesis, we can use a large number for M , for hyperplanes representable using d floats, that would be $M = 2^{32d}$.

1.3 Linear Regression

The Trick

- **Regression:**
 - Features: $x = (x_1, \dots, x_d)$
 - Labels: R
- **Linear Model:**
 - Parameters of hypothesis w : $(w_1, w_2, \dots, w_d, b)$
 - **Predict:** $w(x) = w_1x_1 + w_2x_2 + \dots + w_dx_d + b = x^T w + b$
- **Trick:**
 - Annoying to have special bias parameter b .
 - Make data $(d+1)$ -dimensional and set first coordinate of all data points equal to 1.
 - Then parameters $(w_0, w_1, w_2, \dots, w_d)$.
 - Predict $w(x) = w_0 \cdot 1 + w_1x_1 + w_2x_2 + \dots + w_dx_d = x^T w$.

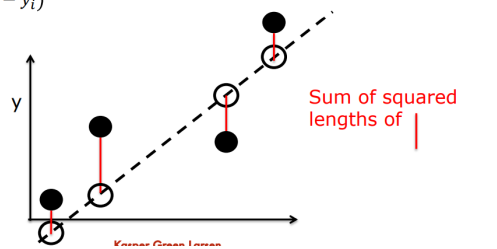
Linear Regression

- **Goal:** Minimize

- $E_{in}(w) = \frac{1}{n} \sum_{i=1}^n (x_i^T w - y_i)^2$

- **Same as minimizing:**

- $\sum_{i=1}^n (x_i^T w - y_i)^2$



Derivative of Matrix

If $l = (l_1, \dots, l_k)$ is a function from $R^d \rightarrow R_k$, then the Jacobian is a matrix of dimension $k \times d$.

- $$\frac{\partial l}{\partial x} = \begin{pmatrix} \frac{\partial}{\partial x_1} l_1(x) & \dots & \frac{\partial}{\partial x_d} l_1(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} l_k(x) & \dots & \frac{\partial}{\partial x_d} l_k(x) \end{pmatrix}$$

Some helpful tricks are:

$$\frac{\partial}{\partial x} Ax + b = A,$$

$$\frac{\partial}{\partial x} x^T x = 2x^T.$$

Linear Regression

- **Goal:** Minimize

- $l(w) = (Xw - y)^T (Xw - y) = \sum_{i=1}^n (x_i^T w - y_i)^2$

- **Compute gradient as transpose of matrix of derivatives.**

- **Matrix of derivatives:**

- $\frac{\partial}{\partial w} (Xw - y)^T (Xw - y) =$

- **Using Chain rule:**

- $2(Xw - y)^T \frac{\partial}{\partial w} (Xw - y) =$

- $2(Xw - y)^T X =$

- $2(w^T X^T X - y^T X).$

- **Gradient as transpose:**

- $2(w^T X^T X - y^T X)^T = 2(X^T X w - X^T y).$

Same as we got earlier

Rules:

- $\frac{\partial}{\partial x} Ax + b = A$

- $\frac{\partial}{\partial x} x^T x = 2x^T$